



SAPIENZA
UNIVERSITÀ DI ROMA

Sapienza-DC: progettazione e sviluppo di un sistema RAG domain-aware

Implementazione di un chatbot multi-dominio basato su Retrieval-Augmented Generation

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica (I3S)
Dipartimento di Ingegneria Informatica Automatica e Gestionale (DIAG)
Corso di Laurea in Ingegneria Informatica e Automatica

Fabio Pastore

Matricola 2129632

Relatore

Prof. Roberto Navigli

Anno Accademico 2025/2026

Sapienza-DC: progettazione e sviluppo di un sistema RAG domain-aware
Tesi di laurea triennale. Sapienza Università di Roma

© 2026 Fabio Pastore. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe Sapthesis.

Email dell'autore: pastore.2129632@studenti.uniroma1.it

To my family and friends.

Sommario

Nella seguenti tesi viene descritta la realizzazione di Sapienza-DC, un chatbot RAG multi-dominio capace di rispondere alle richieste degli utenti mediante analisi e validazione di informazioni estratte da pagine web, fornendo risposte coerenti affiancate da un punteggio di affidabilità nonché dalle fonti utilizzate dall’LLM per rispondere.

I capitoli contenuti in questo elaborato tratteranno gli obiettivi fondamentali di Sapienza-DC, la raccolta dei requisiti di progetto, l’analisi delle tecnologie utilizzate, l’architettura del sistema, la progettazione, la fase di implementazione e gli sviluppi del progetto, proposti in ordine cronologico; la gestione del contesto dell’LLM e un excursus sulle ottimizzazioni introdotte; la sicurezza della pipeline dinanzi a tentativi di prompt injection e una valutazione qualitativa, sotto forma di benchmark, del sistema complessivo.

Indice

1	Introduzione	1
1.1	Presentazione del progetto	1
1.2	Requisiti di progetto	1
2	Analisi delle tecnologie impiegate	3
2.1	Concetti preliminari di NLP	3
2.1.1	Retrieval-Augmented Generation (RAG)	3
2.1.2	Large Language Model (LLM)	3
2.1.3	Bi-encoder, cross-encoder	3
2.2	Framework, strumenti e librerie di sviluppo	3
2.2.1	FastAPI	3
2.2.2	LangChain	3
2.2.3	HuggingFace	3
2.2.4	Ollama e llama.cpp	3
2.2.5	SearXNG	3
2.2.6	Crawl4AI	3
2.2.7	Jinja2	3
2.2.8	TailwindCSS	3
2.3	Containerizzazione ed orchestrazione	3
2.3.1	Architettura a microservizi: Docker e Docker Compose	3
3	Architettura del sistema	5
4	Implementazione della pipeline RAG	7
5	Gestione del contesto e generazione delle risposte	9
6	Ottimizzazioni, sicurezza e benchmark	11
7	Conclusione	13

Capitolo 1

Introduzione

1.1 Presentazione del progetto

Sapienza-DC è stato realizzato come progetto finale per il corso di Laboratorio di Ingegneria Informatica.

In seguito alla realizzazione di un progetto preliminare, il quale consisteva nella realizzazione di un parser per opportuni domini assegnati a ciascun gruppo partecipante, sono stati selezionati i migliori progetti, al fine di avere l'opportunità di accedere ad un progetto personalizzato.

Tra i vari progetti personalizzati veniva proposta la realizzazione di un chatbot di dominio capace di ricevere in input una query da parte dell'utente, selezionare un dominio su cui cercare informazioni per rispondere alla domanda, ricercare fonti su pagine web appartenenti a quest'ultimo, estrarre e processare i contenuti rilevanti da fornire a un LLM avente il compito di rispondere al quesito posto, fornendo aggiuntivamente le fonti di riferimento impiegate.

1.2 Requisiti di progetto

Le specifiche fornite preliminarmente per la realizzazione del progetto consistevano in una descrizione generale dell'architettura della pipeline da implementare:

- consentire all'utente di porre al proprio sistema una query;
- assicurarsi che la query dell'utente possa essere risolta sfruttando i domini disponibili;
- ricercare online pagine (nei domini disponibili) che possano contenere informazioni utili;
- parsare le pagine (sfruttando i parser dell'esonero) e fornire ad un LLM la query assieme ai dati estratti da queste ultime;
- restituire la risposta all'utente.

In aggiunta a quanto appena elencato, si richiedevano inoltre diverse funzionalità aggiuntive. In particolare, il sistema implementato sarebbe dovuto essere capace di: gestire richieste ambigue, chiedendo, ove necessario, chiarimenti all'utente; rigettare la query dell'utente qualora quest'ultima fosse ritenuta inaccettabile o inopportuna; classificare i contenuti delle pagine web per grado di pertinenza alla domanda, valutando eventualmente la possibilità di estrarre dati da domini qualunque in caso di query esterna ai domini assegnati; riassumere o selezionare porzioni delle informazioni ottenute in fase di parsing per evitare di saturare la context window del large language model; citare le fonti e stimare l'affidabilità della risposta fornita, gestendo opportunamente il caso in cui le informazioni in possesso fossero parziali o insufficienti per rispondere alla domanda dell'utente.

Di seguito viene proposto il diagramma utilizzato per presentare i requisiti di progetto.

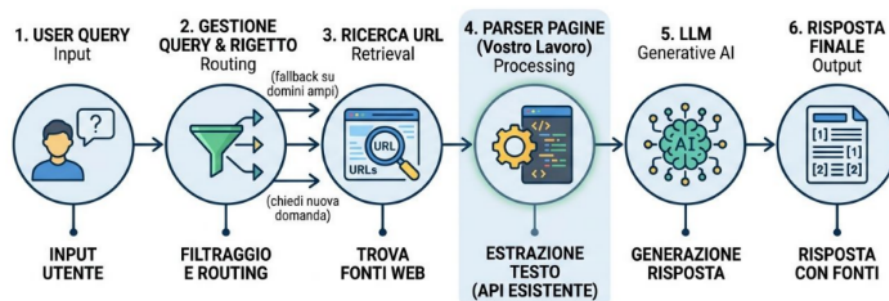


Figura 1.1. Diagramma esemplificativo fornito assieme ai requisiti di progetto.

Si noti come tali specifiche rappresentino solo una linea guida alla realizzazione del progetto, descrivendo solo gli aspetti generali e le funzionalità fondamentali che quest'ultimo dovrebbe implementare.

Questo è voluto: tra gli obiettivi del progetto personalizzato vi è proprio quello di lasciare al gruppo la libertà, e dunque anche le difficoltà che ne conseguono, di produrre quanto richiesto utilizzando gli strumenti nonché tecnologie ritenute più consone.

A tal riguardo, nel capitolo che segue introdurremo le principali tecnologie e framework utilizzati per sviluppare Sapienza-DC.

Capitolo 2

Analisi delle tecnologie impiegate

2.1 Concetti preliminari di NLP

2.1.1 Retrieval-Augmented Generation (RAG)

2.1.2 Large Language Model (LLM)

2.1.3 Bi-encoder, cross-encoder

2.2 Framework, strumenti e librerie di sviluppo

2.2.1 FastAPI

2.2.2 LangChain

2.2.3 HuggingFace

2.2.4 Ollama e llama.cpp

2.2.5 SearXNG

2.2.6 Crawl4AI

2.2.7 Jinja2

2.2.8 TailwindCSS

2.3 Containerizzazione ed orchestrazione

2.3.1 Architettura a microservizi: Docker e Docker Compose

2.4 Database relazionale

2.4.1 MariaDB

Capitolo 3

Architettura del sistema

Your content goes here. If you compile this file, it will run perfectly without any errors, and the official logo will be placed precisely where the University guidelines require.

Capitolo 4

Implementazione della pipeline RAG

Fase implementativa.

Capitolo 5

Gestione del contesto e generazione delle risposte

Capitolo 6

Ottimizzazioni, sicurezza e benchmark

Capitolo 7

Conclusione

